

FAQ Platform

Complete API Reference & Technical Implementation Guide

[Authentication](#) • [Questions](#) • [Answers](#) • [Moderation](#) • [FAQ](#) • [Search](#) • [Reports](#) • [Support Tickets](#) • [Notifications](#) • [Profiles](#)

1. Authentication APIs

Method	Endpoint	Description	Status	Auth/Role
POST	/api/auth/register	Create account	201	Public
POST	/api/auth/login	Login, return access + refresh tokens	200	Public
GET	/api/auth/me	Get current user profile	200	User+
POST	/api/auth/refresh	Rotate refresh token, issue new access token	200	Public
POST	/api/auth/logout	Invalidate refresh token	204	User+
PATCH	/api/auth/password	Change own password	200	User+

2. Category APIs

Categories are the top-level grouping for all questions and FAQs. Every question belongs to a category. These are seeded by admins.

Method	Endpoint	Description	Status	Auth/Role
GET	/api/categories	List all categories with question counts	200	Public
GET	/api/categories/:id	Get single category + its questions	200	Public
POST	/api/categories	Create new category	201	Admin
PATCH	/api/categories/:id	Edit category name / description	200	Admin
DELETE	/api/categories/:id	Delete category (only if no questions)	204	Admin

3. Question APIs

Method	Endpoint	Description	Status	Auth/Role
--------	----------	-------------	--------	-----------

GET	/api/questions	List questions — filter by status/tag/category, cursor pagination	200	Public
POST	/api/questions	Create question (triggers duplicate check)	201	User+
GET	/api/questions/search	Full-text + Fuse.js search across questions & FAQs	200	Public
GET	/api/questions/similar	Duplicate check while typing — returns top matches	200	Public
GET	/api/questions/faqs	List FAQ-only questions (isFAQ: true)	200	Public
GET	/api/questions/:id	Get single question with answers + view count bump	200	Public
PATCH	/api/questions/:id	Edit own question body/tags/category	200	Owner
DELETE	/api/questions/:id	Soft-delete own question (sets status: deleted)	204	Owner/Admin
PATCH	/api/questions/:id/status	Change question status (open/resolved/closed)	200	Admin/Mod
POST	/api/questions/:id/faq	Promote question to FAQ (sets isFAQ: true)	200	Admin
DELETE	/api/questions/:id/faq	Revert FAQ back to resolved question	200	Admin

Query Params for GET /api/questions

```
GET
/api/questions?status=open&category=<id>&tag=javascript&cursor=<lastId>&limit=20&sort=newest

// sort options: newest | oldest | mostViewed | unanswered
```

4. Answer APIs

Method	Endpoint	Description	Status	Auth/Role
GET	/api/questions/:id/answers	List all visible answers for a question	200	Public
POST	/api/questions/:id/answers	Submit answer (enters pending moderation)	201	User+
PATCH	/api/questions/:id/answers/:aid	Edit own answer body	200	Owner
DELETE	/api/questions/:id/answers/:aid	Soft-delete own answer	204	Owner/Admin
POST	/api/questions/:id/answers/:aid/upvote	Upvote an answer (toggle)	200	User+

PATCH	/api/questions/:id/answers/:id/best	Mark as best answer + resolve question	200	Admin/Mod
--------------	-------------------------------------	--	-----	-----------

Answer Schema

```
{
  questionId: ObjectId,    // ref: Question
  body:       String,      // min 30, max 2000 chars
  author:     ObjectId,    // ref: User
  status:     { type: String, enum:
['pending','visible','flagged','under_review','rejected'], default: 'pending' },
  isBestAnswer: { type: Boolean, default: false },
  upvotes:     [ObjectId],  // array of user IDs who upvoted
  upvoteCount: Number,     // denormalised for sort performance
  createdAt:   Date,
  updatedAt:   Date
}

// State machine
pending → visible      (moderation passed)
        → flagged      (moderation failed)
flagged → under_review (admin opens it)
         |
         +→ approved → visible
         +→ rejected
```

5. Moderation APIs

All moderation routes require Moderator or Admin role. Flagged content is hidden from public view until reviewed.

Method	Endpoint	Description	Status	Auth/Role
GET	/api/moderation/queue	Get flagged content queue (AI-priority sorted)	200	Mod+
GET	/api/moderation/queue/answered	Questions with new answer submissions to verify	200	Mod+
GET	/api/moderation/queue/resolved	Resolved questions ready for FAQ promotion	200	Mod+
PATCH	/api/moderation/:targetId/approve	Approve flagged item — restore to visible	200	Mod+
PATCH	/api/moderation/:targetId/reject	Reject flagged item — remove and notify user	200	Mod+
PATCH	/api/moderation/:targetId/escalate	Escalate to senior admin	200	Mod+
GET	/api/moderation/audit-log	Admin audit log of all moderation actions	200	Admin

Moderation Target Schema

```
{
```

```

targetType: { type: String, enum: ['question', 'answer'] },
targetId: ObjectId,
reportedBy: ObjectId, // null if AI-flagged
reason: String,
aiSeverity: { type: String, enum: ['low', 'medium', 'high'] },
status: { type: String, enum: ['pending', 'approved', 'rejected', 'escalated'] },
reviewedBy: ObjectId,
reviewedAt: Date,
createdAt: Date
}

```

6. Search APIs

Unified search runs across FAQs, questions, and tags in a single endpoint. Fuse.js handles fuzzy matching; MongoDB text index handles keyword search. Both results are merged and ranked.

Method	Endpoint	Description	Status	Auth/Role
GET	/api/search	Unified search: FAQs + questions + tags	200	Public
GET	/api/search/tags	Autocomplete tag suggestions	200	Public

Search Query Params

```

GET /api/search?q=admission+deadline&type=faq&category=<id>&tag=deadlines&limit=10

// type: all | faq | question (default: all)
// Response
{
  "success": true,
  "data": {
    "faqs": [...], // isFAQ: true results
    "questions": [...], // open/resolved results
  },
  "meta": { "query": "admission deadline", "totalResults": 12 }
}

```

7. Report APIs

Users report individual questions or answers. AI assigns a severity level. Moderators review and resolve reports from the moderation queue.

Method	Endpoint	Description	Status	Auth/Role
POST	/api/reports	Submit a report on a question or answer	201	User+
GET	/api/reports	List all reports (filterable by status/type)	200	Mod+
GET	/api/reports/:id	Get single report detail	200	Mod+

PATCH	/api/reports/:id/resolve	Mark report as resolved + take action	200	Mod+
PATCH	/api/reports/:id/dismiss	Dismiss report as invalid	200	Mod+

Report Schema

```
{
  targetType: { type: String, enum: ['question','answer'] },
  targetId: ObjectId,
  reportedBy: ObjectId,
  type: { type: String, enum:
['spam','abuse','misinformation','irrelevant','outdated'] },
  description: String, // optional detail from user
  aiSeverity: { type: String, enum: ['low','medium','high'] },
  status: { type: String, enum: ['open','resolved','dismissed'], default: 'open' },
  resolvedBy: ObjectId,
  createdAt: Date
}
```

8. Support Ticket APIs

Support tickets are fully isolated from the FAQ system. They handle sensitive or niche issues. Screenshots upload to Cloudinary. All discussion stays private within the ticket thread.

Method	Endpoint	Description	Status	Auth/Role
GET	/api/tickets	List own tickets (user) or all tickets (mod)	200	User+
POST	/api/tickets	Create support ticket with description	201	User+
GET	/api/tickets/:id	Get ticket detail + thread messages	200	Owner/Mod
PATCH	/api/tickets/:id/status	Update ticket status (in_progress / resolved)	200	Mod+
POST	/api/tickets/:id/messages	Post a message in the ticket thread	201	Owner/Mod
POST	/api/tickets/:id/attachments	Upload screenshot to Cloudinary, attach to ticket	201	Owner
DELETE	/api/tickets/:id/attachments/:aid	Remove an attachment	204	Owner/Mod
PATCH	/api/tickets/:id/assign	Assign ticket to a moderator	200	Admin/Mod

Support Ticket Schema

```
// SupportTicket collection (isolated from FAQ system)
{
  title: String,
  description: String,
  createdBy: ObjectId, // ref: User
}
```

```

assignedTo: ObjectId,          // ref: User (moderator)
status:    { type: String, enum: ['open','in_progress','resolved'], default: 'open' },
attachments: [{
  url:      String,           // Cloudinary URL
  publicId: String,           // Cloudinary public_id for deletion
  uploadedAt: Date
}],
createdAt: Date,
updatedAt: Date
}

// TicketMessage collection (thread replies)
{
  ticketId: ObjectId,          // ref: SupportTicket
  author:   ObjectId,          // ref: User
  body:     String,
  createdAt: Date
}

// Cloudinary upload flow
POST /api/tickets/:id/attachments
→ multer receives file in memory
→ cloudinary.uploader.upload_stream(buffer)
→ store { url, publicId } in ticket.attachments

```

9. Notification APIs

In-app notifications are triggered by key platform events: answer posted on your question, your answer marked as best, moderation decision on your content, ticket reply received.

Method	Endpoint	Description	Status	Auth/Role
GET	/api/notifications	List notifications for current user (paginated)	200	User+
PATCH	/api/notifications/:id/read	Mark single notification as read	200	User+
PATCH	/api/notifications/read-all	Mark all notifications as read	200	User+
DELETE	/api/notifications/:id	Delete a notification	204	User+

Notification Schema + Trigger Events

```

{
  userId:    ObjectId,          // recipient
  type:      { type: String, enum: [
    'answer_posted',           // someone answered your question
    'best_answer_selected',     // your answer was chosen as best
    'answer_approved',         // your flagged answer was approved
    'answer_rejected',         // your answer was rejected
    'ticket_reply',            // moderator replied to your ticket
    'ticket_resolved'          // your ticket was resolved
  ]},
  referenceId: ObjectId,        // question, answer, or ticket ID
  referenceType: String,        // 'question' | 'answer' | 'ticket'
  message:   String,            // human-readable notification text
  isRead:    { type: Boolean, default: false },
  createdAt: Date
}

```

```

}

// Trigger example - after answer is posted:
await Notification.create({
  userId:      question.createdBy,
  type:        'answer_posted',
  referenceId: question._id,
  referenceType: 'question',
  message:     'Someone answered your question: ' + question.title.slice(0, 60)
});

```

10. User Profile APIs

The contributor profile surfaces a user's activity: questions asked, answers given, answers approved as best. Used in Journey 2 (Contributor Profile touchpoint).

Method	Endpoint	Description	Status	Auth/Role
GET	/api/users/:id/profile	Public contributor profile + activity stats	200	Public
GET	/api/users/:id/questions	Questions asked by this user (paginated)	200	Public
GET	/api/users/:id/answers	Answers given by this user (paginated)	200	Public
PATCH	/api/users/me	Update own profile (name, avatar)	200	User+
GET	/api/users	List all users (admin management view)	200	Admin
PATCH	/api/users/:id/role	Change user role (promote to mod / demote)	200	Admin
DELETE	/api/users/:id	Deactivate user account	204	Admin

Profile Response Shape

```

// GET /api/users/:id/profile
{
  "success": true,
  "data": {
    "_id": "...",
    "name": "Riya Sharma",
    "role": "user",
    "joinedAt": "2024-09-01",
    "stats": {
      "questionsAsked": 12,
      "answersGiven": 34,
      "bestAnswers": 8, // answers marked as best
      "upvotesReceived": 91
    }
  }
}

```

11. Standard Response Format

Every endpoint returns the same JSON envelope. This ensures consistent client-side handling and makes the API predictable when used with MiniMax M2.7 or any code-generation model.

```
// Success - single resource
{ "success": true, "data": { /* resource */ } }

// Success - list with pagination
{
  "success": true,
  "data": [ /* array */ ],
  "meta": { "nextCursor": "<lastId>", "hasMore": true, "total": 120 }
}

// Error
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR", // machine-readable constant
    "message": "title is required", // human-readable
    "fields": { "title": "required", "body": "minLength 20" } // validation errors only
  }
}

// Error code constants
VALIDATION_ERROR 400 request body failed validation
UNAUTHORIZED      401 missing or expired JWT
FORBIDDEN         403 authenticated but wrong role
NOT_FOUND         404 resource does not exist
DUPLICATE_QUESTION 409 too similar to an existing question
RATE_LIMITED      429 too many requests in window
INTERNAL_ERROR     500 unhandled server exception
```

12. HTTP Status Codes Reference

Code	Meaning	When to use
201	Created	POST that creates a new resource
200	OK	Successful GET, PATCH, POST returning data
204	No Content	Successful action with no response body (delete, logout)
400	Bad Request	Validation failed — missing fields, wrong types
401	Unauthorized	Missing, expired, or invalid JWT
403	Forbidden	Valid token but wrong role for this route
404	Not Found	Resource ID does not exist
409	Conflict	Duplicate question above similarity threshold
429	Too Many Req	Rate limit exceeded
500	Server Error	Unhandled exception

13. Input Validation

Use express-validator on every mutating endpoint. The same validateRequest helper is reused across all routes.

```
// middleware/validate.js - shared handler
import { validationResult } from 'express-validator';

export const validateRequest = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({
      success: false,
      error: {
        code: 'VALIDATION_ERROR',
        message: 'Validation failed',
        fields: Object.fromEntries(errors.array().map(e => [e.path, e.msg]))
      }
    });
  }
  next();
};
```

Validation Rules by Endpoint

```
// Auth
register:  name(2-50), email(valid), password(min8, lupper, 1digit)
login:    email(valid), password(notEmpty)

// Questions
POST:     title(10-150), body(min20), tags(array 1-5), category(ObjectId)
PATCH:   title(optional,10-150), body(optional,min20), tags(optional)

// Answers
POST:     body(min30, max2000)
PATCH:   body(min30, max2000)

// Reports
POST:     targetType(enum), targetId(ObjectId), type(enum), description(optional,max500)

// Support Tickets
POST:     title(5-100), description(min20, max2000)
Messages: body(min1, max2000)

// Categories
POST/PATCH: name(2-60), description(optional, max200)
```

14. Scalability

Database Indexes

```
// Questions
QuestionSchema.index({ status: 1, createdAt: -1 });
QuestionSchema.index({ category: 1, status: 1 });
QuestionSchema.index({ tags: 1 });
QuestionSchema.index({ isFAQ: 1 });
QuestionSchema.index({ title: 'text', body: 'text' });

// Answers
AnswerSchema.index({ questionId: 1 });
AnswerSchema.index({ questionId: 1, status: 1 });
AnswerSchema.index({ author: 1 });

// Notifications
NotificationSchema.index({ userId: 1, isRead: 1, createdAt: -1 });

// Tickets
TicketSchema.index({ createdBy: 1, status: 1 });
TicketSchema.index({ assignedTo: 1, status: 1 });
TicketMessageSchema.index({ ticketId: 1 });

// Reports
ReportSchema.index({ status: 1, aiSeverity: -1 });
```

Cursor Pagination

```
// All list endpoints use cursor-based pagination
GET /api/questions?cursor=<lastId>&limit=20

const items = await Question
  .find({ _id: { $gt: cursor }, status: 'open' })
  .sort({ _id: 1 })
  .limit(Number(limit) + 1);

const hasMore = items.length > limit;
if (hasMore) items.pop();

res.json({ success: true, data: items,
  meta: { nextCursor: items.at(-1)?._id, hasMore } });
```

Rate Limiting

```
import rateLimit from 'express-rate-limit';

app.use('/api/auth', rateLimit({ windowMs: 15*60*1000, max: 10 }));
app.use('/api/questions', rateLimit({ windowMs: 15*60*1000, max: 30 }));
app.use('/api/tickets', rateLimit({ windowMs: 15*60*1000, max: 20 }));
app.use('/api/reports', rateLimit({ windowMs: 15*60*1000, max: 15 }));
```

15. Recommended Development Order

#	Task	Notes
1	Project setup + MongoDB connection	Schemas, indexes, environment config

2	Authentication	Register, login, /me, refresh, logout
3	Categories	Seed categories, list endpoint
4	Question CRUD	Create, list, detail, edit, delete, view-count
5	AI Duplicate Detection	Fuse.js, /similar endpoint, cache layer
6	Answer System	Post, edit, delete, upvote, best-answer mark
7	Moderation System	Flagged queue, approve/reject/escalate
8	FAQ Conversion	Promote endpoint, FAQ list, revert
9	Search	Unified search across FAQs + questions + tags
10	Report System	Report question/answer, admin resolves
11	Support Ticket System	Create ticket, Cloudinary upload, thread replies
12	Notifications	In-app notifications, mark read
13	User Profiles	Contributor profile, question/answer history
14	Audit Log	Admin action log, filterable by moderator
15	UI optimisation + Deployment	Rate limiting, caching, Atlas production config

Every section of this document maps directly to one or more steps in the order above. Build and test each step before moving to the next — especially moderation before FAQ conversion, since FAQ promotion depends on a working best-answer flow.